



Univerza v Mariboru

---

Fakulteta za elektrotehniko,  
računalništvo in informatiko

# NPO - Kratek pregled projekta

Žiga Pešti, Anja Grudnik, Anže Žunec

Maribor, Junij 2026

# Kazalo

|                                   |   |
|-----------------------------------|---|
| Github povezava do projekta ..... | 3 |
| Opis implementacije .....         | 3 |
| ORV povezava .....                | 4 |
| Mobilna aplikacija .....          | 4 |
| MQTT broker .....                 | 4 |
| Backend in baza .....             | 4 |
| Uporabljene tehnologije .....     | 4 |
| Izgled aplikacije .....           | 6 |
| Prispevki posameznih članov ..... | 7 |

## Github povezava do projekta

<https://github.com/Projektno-delo/Projektna-naloga---Analize-masivnih-podatkov-za-aplikacije-v-realnem-svetu>

## Opis implementacije

Glavni namen je, da mobilna aplikacija zajema GPS lokacijo in podatke pospeškometra, jih pošilja prek MQTT protokola ter omogoča prikaz stanja naprave in meritev v spletnem delu sistema.

Mobilna aplikacija lahko prejme MQTT zahtevo za preverjanje uporabnika s kamero telefona in rezultat posreduje backendu.

Mobilna aplikacija je izdelana v Expo/React Native okolju. Uporabnik se lahko prijavi, nato pa na nadzorni plošči vidi stanje MQTT povezave, deviceId, naslov brokerja, GPS lokacija in trenutne vrednosti pospeškometra. Aplikacija podpira tudi ročno pošiljanje testne meritve v MongoDB podatkovno bazo.

Senzorski del aplikacije deluje v treh glavnih korakih:

- zajem GPS lokacije prek Expo Location,
- zajem pospeškometra prek Expo Sensors,
- pošiljanje meritev prek MQTT na zavihek hribovc/senzorji

Poleg meritev aplikacija pošilja heartbeat sporočila na topic hribovc/heartbeat in statusna sporočila na topic hribovc/status. Heartbeat omogoča, da spletni dashboard zazna, ali je naprava aktivna ali neaktivna. Statusna sporočila omogočajo prikaz online/offline stanja naprave.

Mobilna aplikacija uporablja tudi lokalno zgodovino meritev. Meritve se shranjujejo lokalno, prikazane so na history zaslonu, kjer so dodani tudi časovni filtri in možnost brisanja zgodovine.

Konfiguracija aplikacije ni vezana na trdo zapisane IP naslove. V config.ts se naslov razvojnega strežnika izračuna iz Expo okolja oziroma se nastavi prek okoljske spremenljivke. To olajša testiranje na telefonu, ker morajo telefon, Expo, MQTT broker in backend delovati v istem omrežju.

## ORV povezava

Backend lahko prek MQTT objavi zahtevo za ORV 2FA na topic `hribovc/orv-2fa/request`. Mobilna aplikacija to zahtevo prejme, odpre modul za preverjanje obraza, zahteva dovoljenje za kamero in zajame sliko oziroma preview frame. Slika se pošlje backendu prek API klica, kjer ORV modul izvede preverjanje. Mobilna aplikacija nato prikaže status preverjanja in potrdi ali zavrne 2FA zahtevo.

Ta del povezuje namensko programsko opremo, MQTT komunikacijo in modul računalniškega vida.

## Mobilna aplikacija

Mobilna aplikacija deluje kot MQTT publisher. Zajema podatke iz senzorjev telefona, skrbi za identiteto naprave in pošilja meritve, heartbeat ter status. Pri ORV deluje tudi kot odjemalec MQTT zahtev in kot kamera za preverjanje uporabnika.

## MQTT broker

Za MQTT broker se uporablja Mosquitto. Broker sprejema sporočila mobilne aplikacije in jih posreduje drugim delom sistema.

Konfiguracija odpira dva porta:

- 1883 za navaden MQTT TCP promet,
- 9001 za MQTT prek WebSocket povezave.

WebSocket port je pomemben za spletno aplikacijo, ker se brskalnik na MQTT povezuje prek `ws://localhost:9001`.

## Spletni dashboard

Spletna aplikacija deluje kot MQTT subscriber. Posluša topics `hribovc/senzorji`, `hribovc/heartbeat` in `hribovc/status`. Na dashboardu prikaže seznam naprav, zadnjo meritve, GPS podatke, vrednosti pospeškometra, zadnji status naprave in active/inactive stanje.

## Backend in baza

Backend del sistema sprejema in hrani senzorske podatke v MongoDB. Uporablja se tudi pri ORV 2FA toku, kjer objavi MQTT zahtevo in prejme rezultat preverjanja obraza prek API klicev.

## Uporabljene tehnologije

- Expo SDK 54, za razvoj in zagon mobilne aplikacije prek Expo Go.
- React Native, za uporabniški vmesnik mobilne aplikacije.
- expo-router, za navigacijo med zasloni.
- Expo Location, za zajem GPS lokacije.
- Expo Sensors, za zajem podatkov pospeškometra.
- Expo Camera, za kamero telefona za ORV 2FA preverjanje.
- Expo Haptics, za odziv uporabniškega vmesnika z vibracijami.
- Mosquitto, za MQTT broker.
- Docker Compose, za zagon Mosquitto brokerja v Docker okolju.
- React/Vite spletna aplikacija, za prikaz spletne strani
- Node.js backend in MongoDB, za API, hrambo meritev in povezava z ORV

## Navodila za zagon

### 1. Zagon MQTT brokerja z Docker okoljem

V mapi namenska-programaska-oprema zaženemo:

```
docker compose up -d
```

Docker Compose zazene Mosquitto broker iz slike eclipse-mosquitto:2. Konfiguracija je vezana iz datoteke:

namenska-programaska-oprema/mosquitto/mosquitto.conf

Preverjanje delovanja:

```
docker ps
```

### 2. Zagon mobilne aplikacije

V mapi mobilne aplikacije:

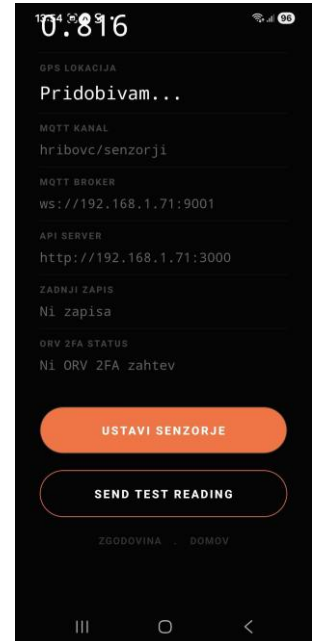
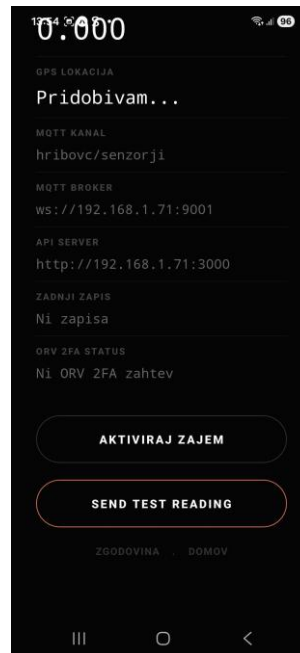
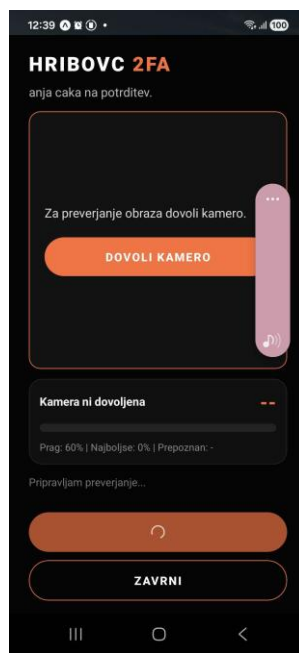
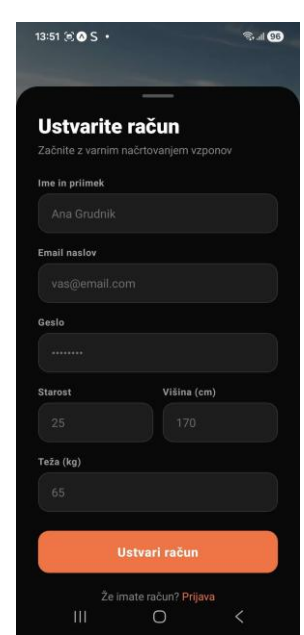
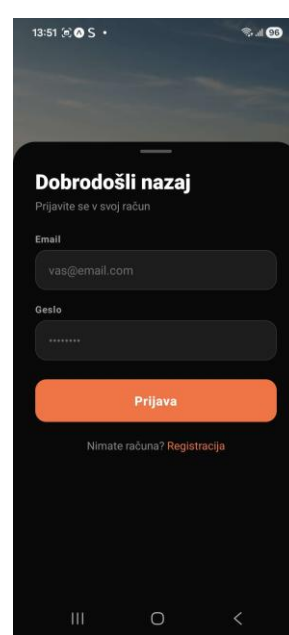
```
cd namenska-programaska-oprema/mobilna-app
```

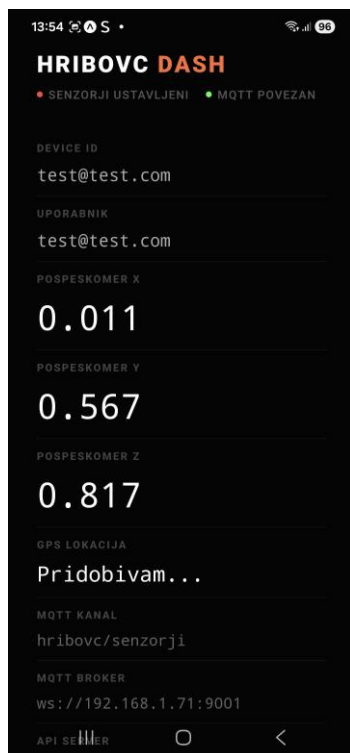
```
npm install
```

```
npx expo start -c
```

Na telefonu mora biti nameščen Expo Go. Telefon in računalnik morata biti v istem Wi-Fi omrežju. V Expo Go se skenira QR koda, ki jo izpise Expo.

## Izgled aplikacije





## Prispevki posameznih članov

### Žiga Pešti

Delal sem predvsem na povezovanju NPO dela z preostalim sistemom, konfiguraciji MQTT komunikacije, backend povezavah ter integraciji ORV modula. Pri projektu sem sodeloval pri prilagoditvi konfiguracije, tako da IP naslovi niso več trdo zapisani, ter vzpostavil trosmerne komunikacije med mobilno aplikacijo, MQTT strežnikom in spletno aplikacijo. Poskrbel sem za nastavitve MQTT infrastrukture, prenos podatkov iz mobilne naprave v backend in podatkovno bazo ter odpravo težav pri shranjevanju senzorskih podatkov. Razvil sem tudi MQTT pomočnika za objavljanje ORV zahtev iz backenda in implementiral podporo za ORV 2FA preverjanje prek mobilnega telefona. V mobilni aplikaciji sem dodal prikaz ORV zahtev v obliki modalnega okna ter poskrbel za združljivost kamere z Expo SDK 54, vključno s podporo za preview in stream način pri ORV prijavi. Poleg tega sem uredil dovoljenja za uporabo kamere na telefonu, pripravil zagonske skripte ter sodeloval pri posodobitvah in izboljšavah projektne dokumentacije.

### Anja Grudnik

Delala sem predvsem na mobilni aplikaciji, uporabniškem vmesniku, zgodovini meritev in MQTT komunikaciji. Pri projektu sem sodelovala pri začetni postavitvi React Native aplikacije ter izdelavi landing, login in register zaslonov. Poskrbela sem za pripravo konfiguracije aplikacije (config.ts) in uporabo dinamičnih IP naslovov. Implementirala sem dashboard, navigacijo ter MQTT komunikacijo, vključno z zajemom GPS lokacije, podatkov pospeškometra in heartbeat sporočili. Razvila sem lokalno shranjevanje zgodovine senzorskih podatkov, history zaslon za pregled meritev ter dodala statistiko meritev, časovne filtre in možnost brisanja zgodovine. Sodelovala sem pri izboljšavah uporabniškega vmesnika, dodala haptic feedback, animacijo pulzirajoče statusne pike ter prikaz prijavljenega uporabnika z mock login funkcionalnostjo. MQTT sporočila sem nadgradila z dodatnimi podatki, kot so deviceId, userEmail in timestamp, ter implementirala status topics in presence payloads za spremljanje stanja naprav. Poleg tega sem pripravila validacijo registracije in Dockerfile za mobilno aplikacijo.

## **Anže Žunec**

Delal sem predvsem na spletnem prikazu MQTT podatkov, zaznavanju aktivnih naprav, MQTT infrastrukturi in integraciji senzorskih podatkov s preostalim sistemom. Spletni MQTT dashboard sem nadgradil za podporo več napravam ter dodal shranjevanje zadnje meritve, heartbeat sporočil in statusnih sporočil za vsako napravo posebej. Implementiral sem izračun active/inactive stanja na podlagi časa zadnjega heartbeat sporočila, periodično osveževanje stanja in štetje vseh trenutno aktivnih naprav. Na dashboardu sem pripravil prikaz seznama naprav, njihovega deviceId, GPS lokacije, podatkov pospeškometra, časa zadnje meritve in online/offline statusa. V mobilni aplikaciji sem dodal prikaz deviceId ter poskrbel, da se pred vzpostavitvijo MQTT povezave naloži identiteta prijavljenega uporabnika. Za MQTT sporočila sem uporabil stabilen deviceId in userEmail ter dodal Last Will sporočilo, ki ob nepričakovani prekinitvi povezave napravo označi kot offline. V backendu sem pripravil endpoint za pregled senzorskih naprav, ki združi meritve in heartbeat podatke iz MongoDB ter izračuna število aktivnih, neaktivnih in vseh naprav. Za MQTT strežnik sem pripravil Docker Compose postavitev Mosquitto brokerja s podporo za navaden MQTT promet na portu 1883 in WebSocket povezavo na portu 9001. Senzorske GPS podatke sem dodatno povezal z načrtovanjem poti v spletni aplikaciji, kjer se lahko zadnja veljavna GPS lokacija prijavljenega uporabnika uporabi kot začetna točka poti, cilj pa se izbere z naslovom ali neposredno na zemljevidu. Poleg tega sem pripravil dokumentacijo NPO sistema z opisom arhitekture, MQTT topicov, postopka zagona, testiranja in odpravljanja pogostih težav.